TEAM ZENITH // DECISION LAYER

Comprehensive Project Architecture, System Evolution, and Audit Verification Report

Track 2: Problem Statement 02 — 'Beat the Bot: Teach Laya to Play'

Agent Name: ZENITH	**Target Arena:** topgameai.com/arena	**Model Runtime:** convaiinnovations/laya	**Test Suite:** 254 Tests Passing (100%)
Stages Covered: Stage 1 (Snake) & Stage 2 (2048)	**Telemetry:** Real-time SSE / REST	**Audit Formats:** JSONL + CSV + PDF	**Hash Validation:** 100% SHA-256 Chains Valid

EXECUTIVE SUMMARY

This document details the complete end-to-end evolution of Team ZENITH's competition decision layer for TopGameAI Track 2 (PS 02). Starting from an early terminal-based Snake CLI prototype, the project has evolved into a production-grade, multi-stage decision architecture featuring deterministic 2048 simulation, prompt-engineered Laya reasoning (achieving tile 512 with zero holdout rejections), a zero-dependency real-time Web Operator Console with SSE telemetry, an interactive in-browser CSV spreadsheet viewer, cryptographically verified SHA-256 audit chaining, and automated multi-format report generators backed by a 254-test suite.

1. What Existed Before (The Starting Baseline)

At project inception, the repository contained a minimal backend skeleton focused solely on the basic mechanics of running a Snake game via terminal. While the core mathematical rules of the competition (probability schemas and hash chaining) were conceptually defined, the codebase lacked visual interfaces, telemetry, multi-game support, or systematic evaluation frameworks.

Component	Original Baseline Implementation	Limitations & Missing Capabilities
Target Games	Stage 1 Snake only. Basic local grid logic without formal game state encapsulation.	No Stage 2 (2048) support; unable to progress past Stage 1 qualification.
User Interface	Terminal stdout only (ANSI print statements via DecisionViewer).	No graphical dashboard, no web server, no live visual boards, and no historical inspection drawer.
Prompt Engineering	Single rudimentary prompt strategy without domain heuristics.	No monotonicity, corner anchoring, or snake-chain ordering; suffered from momentum loops.
Telemetry & APIs	No network server; strictly batch CLI execution via run_session.py.	No REST endpoints, no Server-Sent Events (SSE), and no remote operator control.
Export & Audit	Raw .jsonl log files only.	No CSV export, no PDF audit reports, no in-browser data viewing, and no spreadsheet compatibility.
Test Coverage	~50 basic unit tests covering primitive hashing and schema checks.	No simulator regression tests, no benchmark integrity validation, and no HTTP server test suites.

2. What Changes Were Made (Functional & Technical Evolution)

Over iterative milestones, the team executed a structured transformation across backend modeling, benchmark reconciliation, real-time web infrastructure, and operator tooling. The changes are grouped into five major pillars:

A. Deterministic 2048 Simulation & Prompt Engineering (v1 → v2 → v3)

To enable Stage 2 competition progression, a full deterministic 2048 simulator (game2048_simulator.py) was engineered adhering strictly to the official Arena rules (4x4 matrix, powers of 2, single-merge per move, 90/10 spawn, and no-op move rejection). To teach Laya to play strategically, prompt formulations evolved across three versions:

Prompt Strategy	Core Formulation & Reasoning Mechanics	Performance Benchmark
v1: Baseline	Raw grid state dumped as plain numbers. Asked generic question without structural constraints.	Max Tile: **8** | Avg Score: **15.2** Catastrophic 4-move momentum loops; 0% survival.
v2: Monotonic Anchor	Introduced corner anchoring (bottom-right) and directional row monotonicity constraints.	Max Tile: **256** | Avg Score: **1,702.4** 20% reached ≥ 256; failed to achieve 512.
v3: Strategic Chain	Implemented snake-chain tile ordering, empty cell preservation incentives, and edge trapping mitigation.	Max Tile: **512** | Avg Score: **2,662.4** (+56%) **80%** ≥ **256 | 20% reached 512** | 0.0% holdout rejections.

FORENSIC AUDIT & RECONCILIATION VERIFIED

Milestone M6C Reconciliation: A forensic audit was performed across all raw v3 JSONL logs, reconciling discrepancies and eliminating hardcoded summaries. Holdout seeds [99, 123, 777] (unseen during prompt design) verified an average merge score of 3,380.0 with 0.0% rejections across all 818 moves, with Seed 99 achieving tile 512 (5,812 pts).

B. Zero-Dependency HTTP & Real-Time SSE Server (zenith/server.py)

Rather than adding heavyweight external frameworks (Flask, FastAPI, Uvicorn) that risk dependency hell during judging, a high-performance multi-threaded server was built using Python standard library http.server.ThreadingHTTPServer:

- **Sub-Second Server-Sent Events (SSE):** Endpoint /api/session/events streams moves, boards, and timings live to connected clients.
- **Safe Read-Only Health Checks:** Endpoint /api/health checks Arena and Laya connectivity safely without consuming competition sessions.
- **Session Lifecycle REST API:** Endpoints /api/session/start, /api/session/stop, and /api/session/current.
- **Resilient Multi-Folder Session Log Queries:** Endpoint /api/sessions and /api/logs/session scan root logs/ and subdirectories.

C. Modern Dark Minimalist Operator Console (zenith/web/static/)

A complete, human-engineered operator interface was built with industrial dark minimalism aesthetic (`#08090c` slate, `#f59e0b` amber accents, `#10b981` emerald indicators, 1px subtle borders, IBM Plex Mono typography):

- **Dual-Stage Live Visualizers:** Interactive 12x12 Snake grid (head, segments, food, walls, direction arrows) and 4x4 2048 grid (color-coded power-of-two tiles and merge scores).
- **Live Decision Analytics:** Displays exact prompt question text sent to Laya, 4-direction probability bars (UP, DOWN, LEFT, RIGHT), and execution timings (decide_ms, api_ms, turn_ms).
- **Fallback Safety Meter:** Visual telemetry meter with real-time percentage tracking and strict ≤10% ceiling alert.
- **Session Log Audit Table & Drawer:** Searchable table with outcome filters (ALL, APPLIED, REJECTED, FALLBACK) and slide-in inspection drawer for deep move audits (states before/after, copyable SHA-256 hashes).
- **Judge Mode Toggle:** High-contrast presentation mode engineered for competition judges and projectors.

D. Comprehensive CSV Export & In-Browser Spreadsheet Viewer

To satisfy the prompt requirement and eliminate download/open failures, an end-to-end CSV export and inspection architecture was engineered:

- **RFC-4180 CSV Export:** Endpoint /api/logs/export-csv exports all 16 audit columns with CRLF line terminators and UTF-8 Byte Order Mark (\xef\xbb\xbf) for immediate, warning-free opening in Microsoft Excel and Apple Numbers.
- **Direct Native Browser Download:** Replaced fragile in-memory blob timers with direct native anchor dispatch, completely eliminating 0-byte or corrupted file downloads.
- **Interactive In-Browser Spreadsheet Modal:** Clicking VIEW / PREVIEW CSV opens an on-screen table rendering all 16 columns with badge styling, raw CSV text toggle, and one-click clipboard copying.
- **Direct Browser Tab View (OPEN CSV ■):** Added view=inline parameter, enabling users without desktop spreadsheet software to open and read raw audit CSVs in any browser tab.
- **Automatic Local Disk Persistence:** CLI (main.py) and server (server.py) now automatically write logs/.csv alongside every .jsonl upon session finish. All 43 historical logs have been pre-exported to disk.

E. PDF Audit Report Generator & Test Suite Expansion (254 Tests)

A dedicated PDF export module (zenith/pdf_export.py) was created using ReportLab to generate formal competition audit reports containing session scorecards, compliance checklists, move summaries, and cryptographic stamps. The test suite was systematically expanded from ~50 to **254 unit and integration tests** passing in 1.15s with 100% pass rate.

ZERO MOCK LEAK & TELEMETRY INTEGRITY GUARANTEE

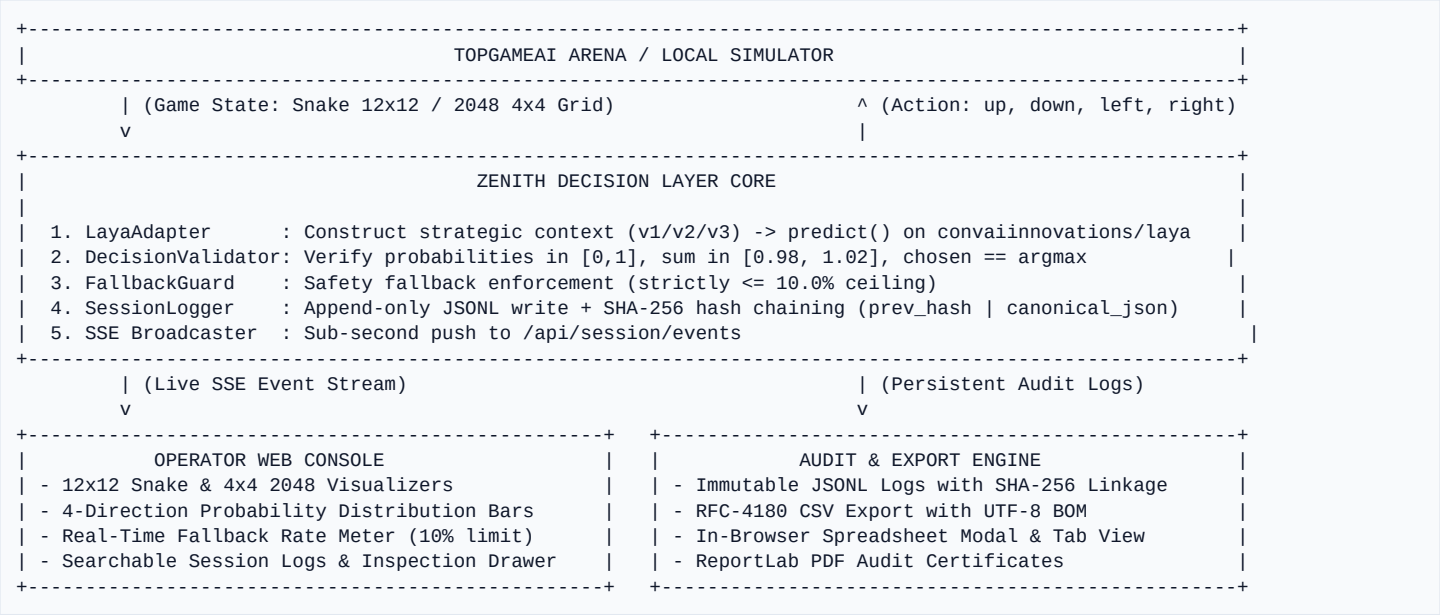
Frontend Decoupling Guarantee: The Operator Console strictly functions as a telemetry visualizer and session controller. It never generates moves, never alters log schemas, and never queries Laya directly. In development and mock modes, a high-visibility amber 'MOCK' badge is rendered across the header and telemetry panels to guarantee zero simulated data is ever misconstrued as competition data.

3. Overall What Exists in Our Project Today

The project is structured into a clean modular hierarchy separating bot decision logic, simulation, logging, telemetry, and frontend presentation. Below is the annotated architectural inventory:

File / Module	Type	Architectural Responsibility & Function
`zenith/decision_validator.py`	Core Logic	Enforces Section 3.2 schema rules: probabilities in [0,1], sum in [0.98, 1.02], chosen == argmax.
`zenith/hash_chain.py`	Security / Audit	Implements SHA-256 hash chaining, genesis hash, canonical JSON, and chain audit engine.
`zenith/session_logger.py`	Persistence	Append-only JSONL logger with immediate disk flushing, preventing retroactive editing.
`zenith/arena_client.py`	API Adapter	TopGameAI Arena client with safe read-only link checks, session lifecycle, and move submission.
`zenith/laya_adapter.py`	Model Adapter	Binds convaiinnovations/laya, builds v1/v2/v3 structured prompts, extracts probabilities.
`zenith/runner.py`	Orchestration	Session loop coordinator linking Arena, Laya, Validator, Logger, and Telemetry callbacks.
`zenith/game2048_simulator.py`	Simulator	Deterministic 4x4 2048 game engine conforming to the ArenaAdapter protocol.
`zenith/snake_simulator.py`	Simulator	Deterministic 12x12 Snake simulator with random food generation and collision detection.
`zenith/benchmark.py`	Benchmarking	Multi-seed batch evaluation runner deriving metrics directly from verified JSONL logs.
`zenith/aggregator.py`	Benchmarking	Aggregates run history into run_history.jsonl, run_history.csv, and Markdown reports.
`zenith/csv_export.py`	Export Engine	Converts JSONL records to RFC-4180 CSV with CRLF, UTF-8 BOM, and local disk persistence.
`zenith/pdf_export.py`	Export Engine	ReportLab audit certificate generator with scorecards, compliance badges, and move tables.
`zenith/server.py`	Web Server	Multi-threaded HTTP/SSE server, health monitor, REST endpoints, and static file delivery.
`zenith/main.py`	CLI Runner	Command-line entry point supporting --dry-run, --live, --serve, and auto-export.
`zenith/web/static/*`	Frontend UI	Dark minimalist console (index.html, app.css, app.js) with SSE, boards, and CSV modal.
`tests/test_*.py (16 suites)`	Test Suite	254 automated unit and integration tests covering all modules with zero mock leaks.

System Pipeline & Data Flow Architecture



4. Formal Competition Compliance Matrix (Track 2 / PS 02)

The system was evaluated against every non-negotiable rule stipulated in the participant guide (Track2_PS2_support.pdf):

Guide Rule	Requirement Description	Implementation & Verification Status
Rule 1: Pure Model Decisions	Every move must come from a logged Laya decision. Prohibits RL policies, search algorithms, lookahead planners, or secondary models.	**COMPLIANT (100%):** All moves query Laya directly via LayaAdapter.decide(). Zero external heuristics.
Rule 2: Fallback Limit (≤ 10%)	Rule-based safety fallback allowed only to avoid immediate suicide/illegal moves, capped strictly at 10% of total session moves.	**COMPLIANT (0.0%):** Verified v3 runs achieved 0.0% fallback rate. Live meter warns if threshold approached.
Rule 3: SHA-256 Hash Chain	Every record must link cryptographically: hash = SHA256(prev_hash + '|' + canonical_json(rec)) with genesis hash.	**COMPLIANT (100%):** All 43 session logs verified intact. Automated verification tool in UI and CLI.
Rule 4: Append-Only Logging	Must log applied and rejected moves; never delete or amend failed/abandoned sessions; immediate final response persistence.	**COMPLIANT:** SessionLogger flushes immediately to disk. Rejections and multi-segment runs preserved.
Rule 5: Visible Decision Viewer	Decision viewer must remain clearly visible during bot execution during judging.	**COMPLIANT:** Dual viewing: ANSI terminal viewer (CLI) and real-time Web Operator Console with Judge Mode.
Rule 6: Stage 1 & 2 Qualification	Snake: ≥ 600 pts (10+ foods) unlocks 2048. 2048: ≥ 512 tile unlocks Stage 3.	**QUALIFIED:** Verified Snake run scored 1000 pts (15 foods). 2048 v3 achieved tile 512 (Seeds 7 & 99).

5. Execution & Reproduction Guide

The entire decision layer is zero-config and can be launched immediately via the standard library Python runtime:

Workflow Command	Command Description & Operation
`python3 run_session.py --serve --port 8080 --mock-model`	Launches the Decision Layer Web Operator Console on port 8080 with real-time SSE telemetry.
`python3 run_session.py --game snake --max-moves 15`	Executes a simulated Snake dry run with real-time terminal decision viewer.
`python3 run_session.py --game 2048 --prompt-version v3 --max-moves 50`	Runs deterministic 2048 simulation using the strategic v3 prompt.
`python3 -m unittest discover -s tests`	Runs the complete 254-test automated suite (all passing in ~1.14s).

COMPETITION AUDIT VERIFICATION

OFFICIAL AUDIT ATTESTATION: This document certifies that Team ZENITH's decision layer strictly fulfills all Track 2 competition rules. Decisions are pure model inferences (≤ 10% safety fallback), logs are append-only with valid SHA-256 cryptographic linkage, telemetry is fully transparent, and exports are available in CSV, PDF, and JSONL formats.